

eBPF Runtime Optimization: Safely Closing the Performance Gap to Native Code

PI: Andi Quinn, Assistant Professor, Computer Science and Engineering, UC Santa Cruz

Grad Student Researcher: Yusheng Zheng, UC Santa Cruz

1 Introduction

eBPF runs on the performance-critical paths of production networking, observability, security, and scheduling systems, where every per-event nanosecond is multiplied across a fleet. eBPF’s safety comes from an in-kernel compilation pipeline in which a userspace compiler lowers each program to bytecode, the kernel verifier proves memory safety and termination [1], and a deliberately simple just-in-time compiler (JIT) translates the verified bytecode to native code one instruction at a time.

Yet, eBPF users pay for verified safety with performance. Our characterization over 27 microbenchmarks extracted from production eBPF programs shows that the same C code runs up to twice as slow through the eBPF pipeline as when compiled directly to native code ($1.57\times$ geomean on x86-64) [7]. The gap is structural and cannot be closed by rewriting bytecode alone. The pipeline lacks hardware idioms (a 64-bit rotate costs 15 machine instructions instead of one `ROL`), profile-guided code layout, and any re-optimization after deployment, when facts such as stable map contents and biased branches become visible. Fixing the JIT in place needs upstream acceptance and a long release cycle, enlarges the trusted computing base (TCB) that years of formal-methods work have hardened [4, 5], and grows per-architecture kernel code.

Related Work: Existing eBPF optimizers (K2 [6], Merlin [3], EPSO [8]) rewrite programs within the instruction set before load, so they cannot exceed what the instruction set can express, and they see no deployment workload. EPASS (a 2025 eBPF Foundation project [2]) transforms programs at load time for programmability and safety within the ISA, while we target the performance gap below the ISA and after deployment. Jitterbug [4] and Agni [5] formally verify the stock JIT and verifier but do not extend the pipeline. We adopt the same philosophy by proving new operations equivalent to verifier-checked bytecode.

2 Closing the Gap, Below the ISA and After Deployment

This project aims to make verified eBPF programs performance-competitive with native code while the stock kernel verifier remains the sole safety authority. The stock verifier re-checks every optimized program before it runs, so no optimizer ever joins the safety argument. Our thesis is that the bytecode the verifier checks is a safety witness, not necessarily the code the CPU must run nor the final form chosen at load time, and each added degree of freedom is paid for with explicit equivalence evidence (re-verification, machine-checked proofs) rather than trust. We therefore deliver optimization infrastructure rather than any single algorithm. Any producer, whether a compiler pass, a runtime policy, or an AI agent, can safely propose optimizations. Three components instantiate this thesis, each relaxing a different coupling in today’s pipeline.

Below the ISA: Kops. Kops [7] is an extension interface, added as a small one-time kernel patch, through which kernel modules add new operations to the pipeline without further core changes. Each operation carries two forms, a proof sequence of standard eBPF instructions that the stock verifier checks on every load, and a native emit of machine instructions that the JIT compiles. A compile-time recognizer rewrites programs to use the operations. Because the verifier checks the proof sequence, neither the recognizer nor the sequence joins the TCB. The native emit is the only per-operation addition, and we discharge that trust with Lean 4 proofs that the emit computes the same result as its proof sequence.

After deployment: BpfReJIT. BpfReJIT re-optimizes deployed programs using facts visible only at runtime. An in-process shim intercepts an unmodified application’s `BPF_PROG_LOAD` and attachment calls. A pass engine then rewrites raw bytecode using runtime facts such as captured map contents and per-site PMU branch profiles, and the shim resubmits every candidate through the stock verifier and JIT, so a rejected candidate leaves the original program running. The same boundary supports load-time specialization and in-place replacement of running programs via stock kernel APIs.

Toward native execution: NativeBPF. The remaining gap to native code motivates the third component. NativeBPF specializes a trusted, faithful ISA simulator to a native program P , producing an eBPF artifact that represents “the simulator executing P ”. The stock verifier analyzes that artifact, and on acceptance the kernel runs P natively. The verified artifact is a safety witness, never executed, extending the decoupling to whole programs without a new native-code checker.

Measurement substrate: an open benchmark. We evaluate all components on our open benchmark built from six production applications (Cilium, Katran, Tracee, Tetragon, BCC, an eBPF continuous profiler), 146 programs, 42 microbenchmarks, built-in correctness oracles, and automated x86-64/ARM64 runs on stock kernels. The suite doubles as a benchmark and harness for AI-agent-driven optimization, its oracles catching unsound transformations from automated search.

Preliminary Results: Kops with seven hardware-idiom operations (rotate, conditional select, bit extract, etc.) speeds up microbenchmarks by up to 24% on x86-64 and 22% on ARM64, recovering 42% of the gap to native code, improves Cilium and Katran datapath throughput by up to 12%, and shrinks generated native code by 12–23%, with no measurable load-time overhead [7]. BpfReJIT transparently applies load-time plans to all six unmodified applications. A NativeBPF native-execution preview running whole-program native replacements reaches $2.358\times$ on Cilium. An LLM-based optimization agent has discovered speedups of up to 34% on suite programs. Every component already exists as a working prototype, including the benchmark suite, the Kops modules and recognizer (preprint [7]), and the BpfReJIT shim and pass engine, all open sourced at <https://github.com/eunomia-bpf/bpf-benchmark>.

Tasks: Preliminary results expose profitability, not mechanism, as the open problem. Applying every matched Kops site can regress ($0.995\times$ on Katran under a coverage-maximizing policy), and a profile-guided branch-layout pass improved one complete six-application run but regressed a rerun. We will therefore pursue: **Task 1:** harden Kops with a cost-model-guided per-site profitability policy, submit the patch series upstream, and drive the community discussion. **Task 2:** extend the operation set beyond hardware idioms to the patterns involving map lookups and helper calls that dominate real datapaths, where idioms alone recover only 5.4% of the achievable gap on Cilium. **Task 3:** build the NativeBPF specialization pipeline and evaluate verifier coverage and native performance against the measured native-code baseline. **Task 4:** complete BpfReJIT: stability-aware gating for profile-guided passes validated on held-out runs; live replacement across links, perf events, tracepoints, XDP, and tail-call program arrays; and phase-change recovery that reverts expired specializations. **Task 5:** harden the existing research repository into a documented, reproducible community benchmark release (packaged workloads, regression tracking, contribution guide).

3 Milestones and Expected Outcomes

Project start: September 1, 2026. **Milestone 1:** December 1, 2026. (Tasks 1, 5) Kops patch series submitted upstream and under discussion; profitability policy evaluated on all six applications; curated benchmark v1.0 release. 1st update blog post.

Milestone 2: May 1, 2027. (Tasks 2, 3) NativeBPF prototype: native extensions accepted by the stock verifier via simulator specialization; extended Kops operation set. 2nd progress update.

Milestone 3: August 1, 2027. (Task 4) BpfReJIT live-replacement coverage and phase-change recovery evaluation. 2nd blog post and paper submissions.

Expected outcomes: open-source releases of the benchmark, Kops modules with Lean 4 proofs, NativeBPF pipeline, and BpfReJIT framework; blog posts; papers at OSDI/SOSP venues; and upstream patch review.

References

- [1] E. Gershuni, N. Amit, A. Gurfinkel, N. Narodytska, J. A. Navas, N. Rinetzky, L. Ryzhyk, and M. Sagiv. Simple and precise static analysis of untrusted linux kernel extensions. In *PLDI '19*, 2019.
- [2] R. Huang. EPASS: Enhancing flexibility and safety of eBPF programs through verifier-cooperative instrumentation. eBPF Foundation Research Grant project, <https://github.com/OrderLab/ePass>, 2025.
- [3] J. Mao, H. Ding, J. Zhai, and S. Ma. Merlin: Multi-tier optimization of ebpf code for performance and compactness. In *ASPLOS '24*, 2024.
- [4] L. Nelson, J. V. Geffen, E. Torlak, and X. Wang. Specification and verification in the field: Applying formal methods to BPF just-in-time compilers in the linux kernel. In *OSDI '20*, 2020.
- [5] H. Vishwanathan, M. Shachnai, S. Narayana, and S. Nagarakatte. Verifying the verifier: ebpf range analysis verification. In *CAV '23*, 2023.
- [6] Q. Xu, M. D. Wong, T. Wagle, S. Narayana, and A. Sivaraman. Synthesizing safe and efficient kernel extensions for packet processing. In *SIGCOMM '21*, 2021.
- [7] Y. Zheng, Z. Ji, W. Tao, H. Sun, W. Zhang, D. Williams, and A. Quinn. Kops: Safely extending the eBPF compilation pipeline with native operations. arXiv preprint arXiv:2606.24213, 2026.

2026.

- [8] Q. Zhu, Y. Liu, Z. Zhu, S. Liu, and L. Bu. Epsos: A caching-based efficient superoptimizer for bpf bytecode. In ASE '25, 2025.